

# Operating System

An **Operating System (OS)** is a special kind of software that works as a **bridge** between the **user** (that's you), the **applications** (like a web browser or a game), and the **hardware** (like your computer's CPU, memory, or hard drive).

## Formal Definition:

An **Operating System** is system software that acts as an **interface** between the **user**, **application programs**, and the **computer hardware**. It manages the hardware and software resources and provides services for computer programs.

## Flow

**User » Application » OS » Hardware**

- **User:** You, the person using the computer.
- **Application:** Programs like Microsoft Word, Chrome, or a game.
- **Operating System:** Software like Windows, Linux, or macOS.
- **Hardware:** Physical parts like CPU, RAM, and hard drive.

## Simple Explanation:

Think of your computer as a restaurant:

- **User:** You're the customer.
- **Application:** The waiter taking your order (like a game or app you use).
- **Operating System:** The kitchen manager who understands both the waiter and the chefs.
- **Hardware:** The chefs, stove, and ingredients.

You don't talk directly to the stove (hardware), right? You tell the waiter (application), and the kitchen manager (OS) makes sure your order is cooked properly by the chefs (hardware).

## Example

Let's say you want to **print a document**:

1. You (user) tell Microsoft Word (application) to print.
2. Word sends a request to the **Operating System**.
3. The OS checks if a printer is connected and available.
4. The OS then tells the **hardware (printer)** to start printing.
5. Your document is printed successfully.

## What if there was no Operating System?

If there were no OS between the **user/application** and the **hardware**, we would face **many serious problems**. Some main issues:

1. **Direct Hardware Control is Very Hard**

**Problem:** Every application would need to know **how to control hardware directly** (keyboard, screen, memory, CPU, etc.)

**Example:** If you're writing a calculator app, you would have to write code that knows **how to handle keyboard signals, draw pixels on the screen, and access memory locations**.

**Why it's a problem:** It's super complex and every app would need to be a mini OS by itself.

## 2. No Memory Management

**Problem:** Applications would not know how to share or organize memory properly.

**Example:** Two programs could accidentally write to the same part of memory and mess up each other's data. Imagine your text editor overwriting the memory used by your music player!

**Why it's a problem:** It can crash programs or corrupt data.

## 3. No Multitasking

**Problem:** Only one program could run at a time because there's nothing to manage sharing the CPU.

**Example:** You wouldn't be able to listen to music and browse the internet at the same time.

**Why it's a problem:** Modern systems need to do many things at once. No OS = no control over task switching.

## 4. No Security or Protection

**Problem:** Any program could access any part of the system, even delete files or spy on user data.

**Example:** A small game app could access your banking data or erase your files without your permission.

**Why it's a problem:** There's no one checking or protecting the system.

## 5. No Standard Rules or Interface

**Problem:** Every program would have to talk to the hardware in its own way. No standard methods.

**Example:** One program might use a printer one way, and another program might use a completely different method.

**Why it's a problem:** Developers would have to write new hardware-handling code for every machine and every situation.

### Summary:

Without an OS, we lose all the management, organization, protection, and standardization it provides. The system would become unusable, unstable, and very difficult to program.

## Processor

Modern processors work in **two modes**:

### User Mode

- Also called non-privileged mode.
- This is the **safe mode** where **normal applications** run.
- Programs running in user mode **can't directly access hardware** or do sensitive operations like controlling memory or devices.
- If they try to, the system will block them or crash the program.

### Example:

When you use a web browser or music player, it's running in **user mode**. It can't directly talk to the keyboard, printer, or memory.

## Kernel Mode

**Kernel Mode** is a special processor mode where the operating system runs with full control over hardware and memory. It is needed for managing important system resources. To protect the system, normal applications run in **user mode**, which has limited access. **System calls** are used to safely switch from user mode to kernel mode when needed.

- Also called privileged mode.
- This is the **powerful mode** where the **operating system** runs.
- It has **full control** over the hardware, memory, CPU, devices, and so on.
- Only **trusted code** like the core part of the OS (called the **kernel**) can run in this mode.

### Example:

When you open a file or connect to Wi-Fi, the actual work (like talking to the disk or network card) is done in **kernel mode**, not by your app directly.

## System Call

A system call is a **secure way** for programs running in **user mode** to request services from the operating system, which runs in **kernel mode**. This separation protects the system from bugs, misuse, or security threats.

## Simple Real-World Analogy:

Imagine you're in a school library.

- You (user program) are not allowed behind the counter (kernel).
- But you want a book (hardware resource).
- So you **fill a request form** (system call) and hand it to the librarian (OS).
- The librarian has the permission to go behind the counter (kernel mode), get the book, and give it to you.

This process protects the library (system) and keeps everything safe and organized.

## Example in Computer Terms:

Suppose a text editor wants to **save a file**:

1. Text editor is in **user mode**
2. It makes a **system call** to "write to disk"
3. The system switches to **kernel mode**
4. The OS writes the data to disk (since only it can do that)
5. Then it switches back to **user mode**

## Why is a System Call Needed?

Because:

- Applications **can't be trusted** to directly access hardware (for safety and stability)
- We need a **controlled way** to allow safe interaction between apps and the system
- It helps the OS **manage everything properly** without risking crashes or security issues